

YOLO 推理服务实践项目:Docker + GitHub + CI

面向 ML/CV 岗位面试的动手练习。做完你会得到一个可以放进简历的 GitHub 仓库,同时把面试常考的 Docker 和 Git/CI 知识点都过一遍。

目标产物:一个用 FastAPI 封装的 YOLOv8 目标检测服务,容器化后能本地运行,代码托管在 GitHub,并且有一条自动跑测试和构建镜像的 CI 流水线。

最终项目结构

```
yolo-inference-service/
├── .github/
│   └── workflows/
│       └── ci.yml
├── app/
│   ├── __init__.py
│   ├── main.py
│   └── detector.py
├── tests/
│   └── test_api.py
├── .dockerignore
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

阶段 0:环境准备

1. 安装 Docker Desktop(Windows/Mac)或 Docker Engine(Linux),装完跑一下确认:

```
bash

docker --version
docker run hello-world
```

2. 确认 Git 已安装,配置好身份:

```
bash

git --version
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
```

3. 登录 GitHub,先不用建仓库,后面阶段 3 再建。

阶段 1:写推理服务

先在本地把服务跑起来,确认逻辑对,再去容器化。不要一上来就 Docker,先让代码能跑。

新建项目目录,然后创建以下文件。

requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
ultralytics==8.3.0
pillow==10.4.0
python-multipart==0.0.12
```

app/init.py

空文件即可,作用是把 app 标记成一个 Python 包。

app/detector.py

python

```
from ultralytics import YOLO
from PIL import Image

class Detector:
    def __init__(self, model_path: str = "yolov8n.pt"):
        # yolov8n 是最小的模型,首次运行会自动下载权重
        self.model = YOLO(model_path)

    def predict(self, image: Image.Image):
        results = self.model(image)
        detections = []
        for r in results:
            for box in r.boxes:
                detections.append({
                    "class": self.model.names[int(box.cls)],
                    "confidence": round(float(box.conf), 4),
                    "bbox": [round(float(x), 2) for x in box.xyxy[0]],
                })
        return detections
```

app/main.py

python

```
import io
from fastapi import FastAPI, File, UploadFile
from PIL import Image
from app.detector import Detector

app = FastAPI(title="YOLO Inference Service")

# 懒加载:启动时不加载模型,第一次请求 /predict 才加载
# 这样 /health 很快,测试时也容易 mock
_detector = None

def get_detector() -> Detector:
    global _detector
    if _detector is None:
        _detector = Detector()
    return _detector

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    image_bytes = await file.read()
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    detections = get_detector().predict(image)
    return {"detections": detections, "count": len(detections)}
```

本地跑起来

```
bash
```

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

打开浏览器访问 (<http://localhost:8000/docs>), FastAPI 自带的 Swagger 界面。在 </predict> 上点 "Try it out", 上传一张随便的照片, 看返回的检测结果。

这一阶段要理解的点:

- 为什么用懒加载而不是在启动时加载模型。提示: 健康检查要快、测试要好 mock、启动失败和推理失败要分开。
- yolov8n/s/m/l/x 的区别(速度和精度的权衡), 你简历里做过类似取舍。

对应面试问题:

- 怎么把一个训练好的模型变成一个线上服务?
- 模型加载放在哪比较好, 启动时还是请求时, 各有什么代价?

阶段 2: 容器化(Docker 核心)

这是 Docker 部分的重点。先写一个最直白的 Dockerfile, 跑通, 再在阶段 5 优化。

.dockerignore

```
__pycache__/  
*.pyc  
.git/  
.github/  
tests/  
*.md  
.venv/  
venv/
```

```
*.pt  
.pytest_cache/
```

`.dockerignore` 决定哪些文件不进构建上下文。漏掉它会把 .git、虚拟环境、模型权重一起塞进镜像,既慢又大。这是面试常被追问的点。

Dockerfile

```
dockerfile  
  
FROM python:3.11-slim  
  
WORKDIR /app  
  
# ultralytics / opencv 需要的系统库  
RUN apt-get update && apt-get install -y --no-install-recommends \  
    libgl1 libglu1-mesa \\  
    && rm -rf /var/lib/apt/lists/*  
  
# 先只拷贝 requirements,利用 Docker 层缓存  
# 只要 requirements 不变,这一层就不重新装依赖  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
# 再拷贝代码  
COPY app/ ./app/  
  
EXPOSE 8000  
  
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

构建和运行

```
bash
```

```
# 构建镜像
docker build -t yolo-inference .

# 运行容器,把容器 8000 端口映射到本机 8000
docker run -p 8000:8000 yolo-inference

# 后台运行并命名
docker run -d -p 8000:8000 --name yolo yolo-inference
```

跑起来后同样访问 `http://localhost:8000/docs` 测试。

一定要练熟的命令

```
bash

docker ps                # 看正在运行的容器
docker ps -a             # 看所有容器,包括停止的
docker images            # 看本地镜像
docker logs yolo         # 看容器日志
docker logs -f yolo      # 实时跟踪日志
docker exec -it yolo bash # 进到容器里面看
docker stop yolo         # 停止
docker rm yolo           # 删除容器
docker rmi yolo-inference # 删除镜像
```

这一阶段要理解的点:

- 镜像(image)和容器(container)的区别。镜像是模板,容器是运行实例。
- 为什么先 COPY requirements.txt 再装依赖,而不是一次性 COPY 全部。答案是层缓存:改代码不应该触发重新装依赖。
- `-p 8000:8000` 左边是宿主机端口,右边是容器端口,别搞反。
- 这个镜像会很大(torch 拖进来好几个 G)。记下镜像大小 `docker images`,阶段 5 会优化。

对应面试问题:

- 镜像和容器有什么区别?
 - Dockerfile 每一行都会产生什么?为什么指令顺序会影响构建速度?
 - `COPY` 和 `ADD` 的区别?
 - 容器里的进程为什么默认是 root,有什么风险?(阶段 5 解决)
 - 数据怎么持久化?容器删了数据还在吗?(引出 volume,见阶段 5)
-

阶段 3:Git 和 GitHub 工作流

现在把代码托管起来,练分支、提交、PR 这套真实协作流程。

.gitignore

```
__pycache__/  
*.pyc  
.venv/  
venv/  
*.pt  
.pytest_cache/  
.DS_Store  
*.egg-info/
```

模型权重(.pt)不进 Git。大二进制文件不应该提交到普通仓库,这是常识题。

初始化并提交

```
bash  
  
git init  
git add .  
git commit -m "feat: initial YOLO inference service"
```

提交信息建议用约定式提交(Conventional Commits)风格:`feat:` `fix:` `docs:` `test:` `chore:`。面试官看你的 commit history 能看出你是否专业。

推到 GitHub

在 GitHub 网页上新建一个空仓库(不要勾选 README,避免冲突),命名 `yolo-inference-service`,然后:

```
bash

git remote add origin git@github.com:你的用户名/yolo-inference-service.git
git branch -M main
git push -u origin main
```

练一遍分支 + PR 流程

这是协作的核心,务必动手走一遍:

```
bash

# 1. 开一个功能分支
git checkout -b feature/add-readme

# 2. 创建 README.md(内容见下方阶段说明),提交
git add README.md
git commit -m "docs: add project readme"

# 3. 推送分支
git push -u origin feature/add-readme
```

然后去 GitHub 网页,你会看到提示创建 Pull Request。创建 PR,自己 review 一下 diff,合并(merge)。合并后回到本地:

```
bash

git checkout main
git pull
git branch -d feature/add-readme    # 删掉本地已合并的分支
```

故意制造一次冲突并解决

面试很爱考解冲突。自己造一个练手:

```
bash

# 在 main 上改 README 第一行,提交
# 再开个分支,也改 README 同一行,提交
# 合并时就会冲突,手动编辑 <<<<<< ===== >>>>>> 标记区域,保留你要的内容
git add README.md
git commit
```

这一阶段要理解的点:

- `git merge` 和 `git rebase` 的区别。merge 保留分叉历史并产生一个合并提交,rebase 把你的提交"挪"到目标分支顶端让历史变直。
- `git fetch` 和 `git pull` 的区别。pull = fetch + merge。
- 什么时候不能 rebase(已经 push 给别人的公共分支)。

对应面试问题:

- 描述一下你日常的 Git 协作流程。
- merge 和 rebase 区别,你团队用哪种?
- 遇到合并冲突怎么处理?
- 不小心把东西提交错了怎么回退?(`git revert` vs `git reset`,前者安全)

阶段 4:GitHub Actions CI

让 GitHub 在每次 push 和 PR 时自动跑测试、构建镜像。这部分越来越常被问。

先写一个测试

tests/test_api.py

```
python

from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

因为用了懒加载,这个测试不会触发模型下载,所以在 CI 里跑得很快。

.github/workflows/ci.yml

```
yaml
```

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest httpx
      - name: Run tests
        run: pytest -v

  docker-build:
    runs-on: ubuntu-latest
    needs: test          # 测试通过了才构建镜像
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t yolo-inference:ci .
```

提交并推送后,去 GitHub 仓库的 Actions 标签页看流水线跑起来。绿勾代表通过,红叉点进去看日志排错。

这一阶段要理解的点:

- `on` 定义触发条件,`jobs` 是并行的任务,job 里的 `steps` 是顺序执行。
- `needs: test` 让 docker-build 依赖 test,形成先后顺序。
- CI 里装 ultralytics 会比较慢(torch 很大),真实项目会用依赖缓存(actions/cache 或 setup-python 的 cache 选项)加速。可以作为优化点提一句。

对应面试问题:

- CI 和 CD 分别指什么?
- 你的流水线里跑哪些检查?(lint、测试、构建、安全扫描)
- 测试一个依赖大模型的接口,怎么避免在 CI 里真的下载模型跑推理?提示:mock 掉 detector,只验证接口逻辑和数据流。

进阶测试:mock 掉模型

这个能体现你懂测试设计。在 tests 里加:

```
python
```

```
def test_predict_mocked(monkeypatch):
    import app.main as main

    class FakeDetector:
        def predict(self, image):
            return [{"class": "person", "confidence": 0.9, "bbox": [0, 0, 10, 10]}]

    monkeypatch.setattr(main, "get_detector", lambda: FakeDetector())

    import io
    from PIL import Image
    buf = io.BytesIO()
    Image.new("RGB", (32, 32)).save(buf, format="JPEG")
    buf.seek(0)

    response = client.post("/predict", files={"file": ("t.jpg", buf, "image/jpeg")})
    assert response.status_code == 200
    assert response.json()["count"] == 1
```

阶段 5:进阶优化(拉开差距的地方)

这些是初级和有经验的人的分水岭,面试加分项。

5.1 多阶段构建 + 非 root 用户

dockerfile

```

# ---- 构建阶段 ----
FROM python:3.11-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---- 运行阶段 ----
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgl1 libglu1-mesa && rm -rf /var/lib/apt/lists/*

# 只把装好的依赖从构建阶段拷过来, 不带编译工具
COPY --from=builder /install /usr/local
COPY app/ ./app/

# 创建非特权用户, 不用 root 跑
RUN useradd -m appuser
USER appuser

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

要点:多阶段构建把"编译时才需要的东西"挡在最终镜像外面,减小体积和攻击面;`USER appuser`让容器不以 root 运行,这是安全基本功。

5.2 给 ML 镜像瘦身:装 CPU 版 torch

ultralytics 默认会拖进带 CUDA 的 torch,几个 G。如果是 CPU 推理服务,可以只装 CPU 版:

```
dockerfile
```

```
RUN pip install --no-cache-dir torch --index-url https://download.pytorch.org/w
  && pip install --no-cache-dir -r requirements.txt
```

构建前后用 `docker images` 对比体积,这个数字面试时能直接说出来很有说服力。

5.3 docker-compose

docker-compose.yml

```
yaml

services:
  inference:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./app:/app/app          # 改代码不用重新构建镜像
    environment:
      - MODEL_PATH=yolov8n.pt
```

```
bash

docker compose up --build
docker compose down
```

要点:compose 用一个文件描述服务、端口、卷、环境变量;volume 把宿主目录挂进容器,实现持久化和热更新。

对应面试问题:

- 多阶段构建解决什么问题?
- 怎么减小一个 Python/ML 镜像的体积?(slim 基础镜、多阶段、CPU 版依赖、.dockerignore、合并 RUN 层、清理 apt 缓存)
- bind mount 和 named volume 区别?

- 为什么不该用 root 跑容器？

阶段 6:写好 README(作品的门面)

README.md 是面试官点开仓库第一眼看到的东西。建议包含:

- 项目一句话简介(用 FastAPI 封装 YOLOv8 的目标检测推理服务)
- 快速开始:本地运行和 Docker 运行两种方式的命令
- 接口说明:/health 和 /predict 的请求和返回示例
- 一个 curl 调用示例:

```
bash

curl -X POST http://localhost:8000/predict \
-F "file=@your_image.jpg"
```

- 技术栈和你做过的优化(多阶段构建、镜像瘦身、CI)

把你做的优化写出来,这正好对应你简历里"工程落地"的能力。

学习顺序建议

1. 阶段 1 到 4 先全部跑通,拿到一个绿色 CI 的仓库。这一步能应付大部分基础提问。
2. 再回头做阶段 5 的优化,这是区分度所在。
3. 全程把每个阶段末尾的"面试问题"用自己的话讲一遍,讲不清楚的地方就是没真懂。

做完之后如果想检验效果,可以让我用阶段里这些面试问题对你做一轮模拟问答,我会追问细节看你是不是真的理解了。